

Hoja informativa: Algoritmos de machine learning

Práctica

```
# estandarizar características
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler() # crear un objeto de clase scaler
scaler.fit(X) # entrenar el estandarizador
X_sc = scaler.transform(X) # transformar el dataset
```

```
# calcular la matriz de correlación
cm = df.corr()
```

```
# iniciar los modelos de regresión lineal
from sklearn import linear_model
linear_regression = linear_model.LinearRegression()
# un modelo de regresión lineal con regularización de peso entero
lasso = linear_model.Lasso()
# un modelo de regresión lineal con regularización de peso entero
ridge = linear_model.Ridge()

# entrenar un modelo
linear_regression.fit(X_train, y_train)
# obtener predicciones
y_pred = linear_regression.predict(X_val)
# imprimir los coeficientes del modelo lineal
print(model.coef_)
# imprimir los coeficientes nulos de regresión
print(model.intercept_)
```

```
# calcular el valor de EAM para un modelo entrenado
# ECM (mean_squared_error) y R2 (r2_score) se encuentran de mane
from sklearn.metrics import mean_absolute_error

print('MAE:', mean_absolute_error(y_true, y_pred))
```

```
# iniciar un modelo de regresión logística (algoritmo para clas:
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

# obtener la probabilidad de que un objeto entre en una clase es
y_probab = model.predict_proba(X_test)
```

```
# construir una matriz de confusión para la clasificación
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)
tn, fp, fn, tp = cm.ravel() # "igualar" la matriz para conseguir
```

```
# calcular las métricas de clasificación
from sklearn.metrics import accuracy_score, precision_score, re
acc = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
roc_auc = roc_auc_score(y_true, probabilities[:,1])
```

```
# entrenar un clasificador de árbol de decisión

tree_model = DecisionTreeClassifier()
tree_model.fit(X_train, y_train)
```

```
y_pred = tree_model.fit(X_test)
```

```
# visualizar un árbol de decisión entrenado
```

```
plt.figure(figsize = (20,15)) # configurar el tamaño de la figura  
plot_tree(tree_model, filled=True, feature_names = X_train.columns)  
plt.show()
```

```
# entrenar un regressor de bosque aleatorio  
from sklearn.ensemble import RandomForestRegressor
```

```
# n_estimators : el número de árboles de decisión  
rf_model = RandomForestRegressor(n_estimators = 100)  
rf_model.fit(X_train, y_train)  
y_pred = rf_model.predict(X_val)
```

```
# entrenar un regresor de potenciación del gradiente  
from sklearn.ensemble import GradientBoostingRegressor
```

```
# n_estimators: el número de modelos simples  
gb_model = GradientBoostingRegressor(n_estimators = 100)  
gb_model.fit(X_train, y_train)  
y_pred = gb_model.predict(X_val)
```

```
# clustering con KMeans  
from sklearn.cluster import KMeans
```

```
# estandarización obligatoria de los datos antes de pasarlo al algoritmo  
sc = StandardScaler()  
X_sc = sc.fit_transform(X)
```

```
km = KMeans(n_clusters = 5) # establecer el número de clústeres  
labels = km.fit_predict(X_sc) # aplicar el algoritmo a los datos
```

```
# trazar un dendrograma
from scipy.cluster.hierarchy import dendrogram, linkage

# estandarización obligatoria de los datos antes de pasarlo al algoritmo
sc = StandardScaler()
X_sc = sc.fit_transform(X)

linked = linkage(X_sc, method = 'ward')

plt.figure(figsize=(15, 10))
dendrogram(linked, orientation='top')
plt.title('Agrupación jerárquica para GYM')
plt.show()

# estimar el valor de la silueta en clustering
from sklearn.metrics import silhouette_score

silhouette_score(x_sc, labels)
```

Teoría

El **mínimo global** de una función es el valor más pequeño de la función.

El **mínimo local** de una función es el valor más pequeño dentro de un rango dado de la función.

El **descenso de gradiente** es un método para encontrar un mínimo local.

Un **gradiente** es un vector formado por derivadas locales.

Los **hiperparámetros** de un modelo son los parámetros cuyos valores se establecen antes de que comience el entrenamiento; no cambian. Un modelo puede tener varios hiperparámetros.

La **normalización** consiste en convertir los valores de característica al rango de 0 a 1.

La **estandarización** consiste en convertir los valores de característica de modo que sean valores aleatorios de la distribución normal estándar (con una expectativa

matemática de 0 y una varianza de 1).

La **multicolinealidad** es una fuerte correlación entre varias características.

La **regularización** se refiere a cualquier limitación adicional de un modelo o a una acción que disminuirá la complejidad del modelo y minimizará el impacto del sobreajuste. Esta información puede ser una "multa" por la complejidad del modelo.